

Documentation et Présentation

Simulation d'épidémie : Py-Démie

Groupe : Kylian STRUB, Mathéo ROSE, Thomas VUILLIN

Objectif initial – Idée de départ

L'objectif initial du projet **Py-Démie** était de concevoir une simulation d'épidémie à l'aide du langage **Python**, afin d'observer la propagation et l'évolution d'un virus au sein d'une population.

Pour cela, le projet devait s'appuyer sur le **modèle mathématique S-I-R (Susceptible – Infecté – Rétabli)**, un modèle épidémiologique classique permettant de représenter les transitions entre différents états de santé des individus au cours du temps.

Même si nous ne connaissions pas très bien ce modèle au départ, nous avons décidé de nous lancer dans ce projet afin d'en apprendre davantage.

En premier lieu, nous avons défini les règles principales de la simulation ainsi que la direction générale du projet. Nous avons choisi de partir sur une **carte en 2D**, contenant des individus représentés sous forme de points de différentes couleurs.

Nous avons également envisagé de créer un **environnement en 3D** pour représenter notre « monde », mais à cause d'un manque de compétence certain dans ce domaine, cette idée a finalement été abandonnée.

La **première version** du jeu était un simple ensemble de points se baladant aléatoirement et transmettant une maladie par contact.

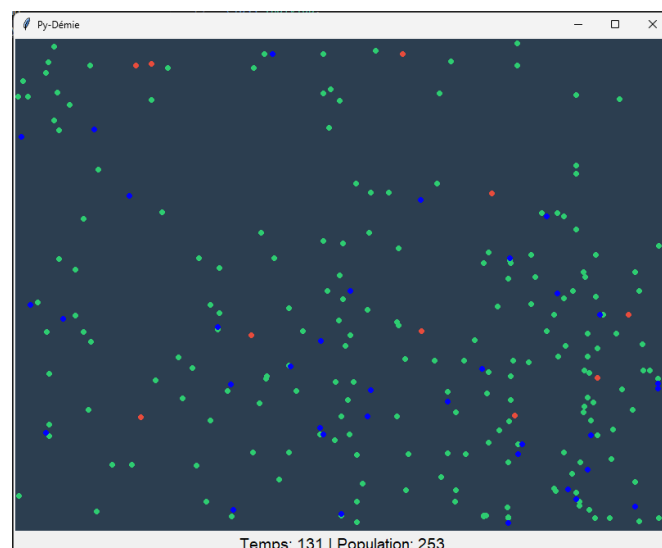


Illustration 1: Première version de Py-Démie

Les éléments mathématiques

Nos premières réflexions ont porté sur les éléments mathématiques que nous pouvions intégrer au projet. Dans un premier temps, il y a évidemment le modèle **SIR**, qui permet de créer un monde **stochastique**, donc non déterministe. Cela nous a permis d'utiliser des outils comme la **loi de Bernoulli**, des tirages aléatoires ou encore des calculs de moyennes afin de simuler la propagation du virus.

Cependant, cela ne nous semblait pas suffisant pour justifier pleinement un projet de mathématiques. Nous avons donc décidé d'ajouter une interface visuelle et des déplacements plus complexes. Cela nous a permis d'exploiter la **géométrie vectorielle**. Tous les déplacements à l'écran reposent sur ce principe : par exemple, lorsqu'un individu se déplace, un **vecteur directeur** est calculé. Nous utilisons également la **distance euclidienne**, notamment pour savoir si un agent a atteint sa destination ou pour vérifier la proximité entre deux individus lors d'une infection.

Enfin, la dernière idée, arrivée plus tard dans le projet, a été d'intégrer la **théorie des graphes**. Cela a permis de rendre la simulation plus réaliste en modélisant un réseau routier. Grâce à cela, nous avons pu calculer des **plus courts chemins**, notamment à l'aide de l'algorithme de **Dijkstra**.

1. Le modèle SIR

La simulation repose sur une implémentation du modèle SIR (Sains-Infectés-Rétablis). Dans le code, chaque individu est une instance de classe possédant un attribut d'état qui évolue selon une machine à états. Nous avons complexifié le modèle théorique de base pour le rendre plus réaliste en y ajoutant les stades **E (Exposé)** pour gérer la période d'incubation, **D (Décédé)** pour la mortalité, et **V (Vacciné)**. Les transitions entre ces états ne sont pas instantanées mais régies par des compteurs temporels (comme *days_infected*) incrémentés à chaque cycle de la simulation, ce qui permet de reproduire fidèlement la cinétique de la maladie.

2. La géométrie vectorielle

Pour assurer des déplacements fluides et réalistes, nous avons exploité la géométrie vectorielle. Chaque déplacement est calculé en déterminant d'abord le vecteur de direction entre la position actuelle de l'agent et sa cible (*dx et dy*). Nous utilisons ensuite le théorème de Pythagore (*via math.hypot*) pour calculer la distance réelle.

Le mouvement est appliqué en normalisant ce vecteur (division par la distance) pour obtenir une direction unitaire, que nous multiplions ensuite par la vitesse de l'agent. Cela garantit que les personnages se déplacent à une vitesse constante quelle que soit la distance à parcourir ou l'angle de la trajectoire.

Enfin, la gestion des interactions (entrée dans un bâtiment, zones de travail) repose sur la détection de collisions géométriques simples : nous vérifions à chaque instant si les coordonnées

(x, y) d'un agent sont contenues dans les rectangles (*rect*) définissant les différentes zones (**Supermarché, Centre Médical, Ville**), ce qui déclenche les changements de comportement appropriés.

3. La théorie des graphes

Pour modéliser le réseau routier de la simulation, nous avons fait appel à la théorie des graphes. La carte de la ville est abstraite sous la forme d'un graphe pondéré non orienté, où chaque nœud représente une intersection ou un point d'intérêt stratégique (*coordonnées x, y*), et chaque arête symbolise un tronçon de route reliant deux points.

Lors de l'initialisation (*NavigationGraph.__init__*), nous construisons une liste d'adjacence qui associe à chaque nœud ses voisins accessibles, le "poids" de la connexion étant la distance euclidienne réelle entre eux.

Pour le déplacement des agents, nous avons implémenté l'algorithme de Dijkstra. Lorsqu'un personnage doit se rendre d'un point A à un point B (par exemple, de son domicile au centre commercial), l'algorithme explore le graphe à l'aide d'une file de priorité (*heapq*) pour déterminer le chemin le plus court en minimisant la distance totale parcourue. Cette approche garantit que les citoyens empruntent toujours l'itinéraire le plus optimal.

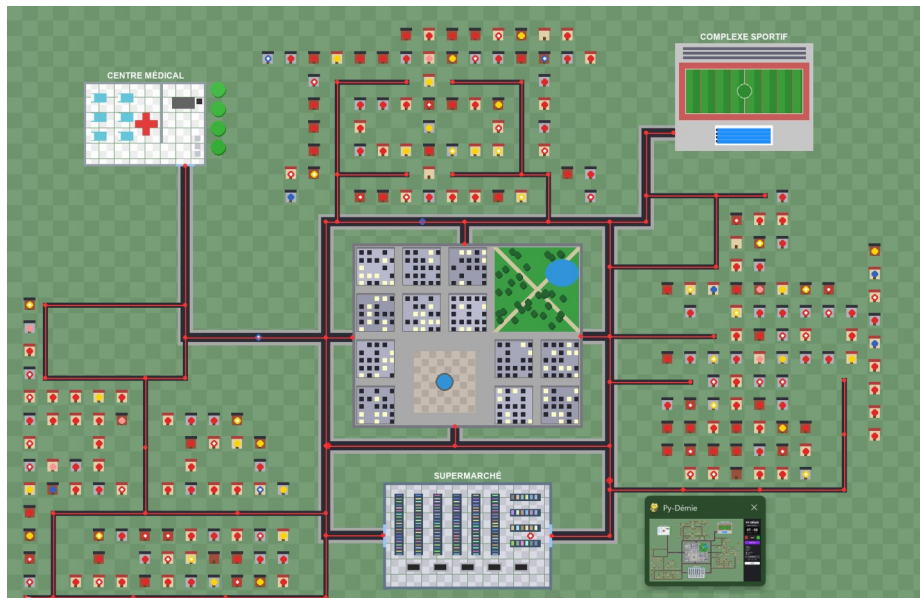


Illustration 2: Tracé rouge représentant le graphe

Documentation générale de la simulation

1. La population et son comportement

La simulation repose sur une population d'agents autonomes hétérogènes. Chaque individu est une instance de la classe Person (fichier [person.py](#)), initialisée avec des propriétés uniques générées via des **lois de probabilité** pour garantir une diversité réaliste.

Lors de la création de la population (`__init__`), nous attribuons à chaque agent un "génom social" aléatoire :

L'âge est tiré selon une loi uniforme (`random.randint(18, 90)`), simulant une population adulte active et retraitée. **L'emploi** est une épreuve de Bernoulli détermine le statut actif de l'individu, avec une probabilité $p=0.6$ (`random.random() < 0.6`), reflétant un taux d'emploi de 60%.

La fragilité est un facteur important pour le modèle épidémiologique. Un agent est considéré comme "fragile" s'il a plus de 70 ans OU si un tirage aléatoire supplémentaire l'indique (`FRAGILITY_RATE`), modélisant les comorbidités invisibles.

Comportement : 20% de la population est marquée comme "sportive", ce qui ajoutera des déplacements spécifiques à leur emploi du temps hebdomadaire.

Le "cerveau" de chaque agent réside dans la méthode `update_behavior`. À chaque itération, l'agent évalue sa situation selon une machine à états hiérarchique priorisée par l'heure de la journée et le jour de la semaine :

1. Impératifs Biologiques : Si l'agent est malade (État D pour Décédé), il ne bouge plus.

2. Vaccination : Priorité absolue si l'agent a décidé de se faire vacciner ce jour-là.

3. Activités Hebdomadaires

Courses : Un jour fixe par semaine (`shopping_day`), l'agent se rend au supermarché.

Sport : Pour les sportifs, un autre jour fixe est alloué à l'activité physique, distinct du jour de courses pour maximiser la distribution des déplacements.

4. Routine Journalière (8h - 19h)

Les Actifs se rendent sur leur lieu de travail (Centre ville ou bâtiments spécifiques s'ils ont un Job assigné via `jobs.py`). Les Inactifs ont une probabilité de 20% de se rendre en ville pour se balader, ou de 50% de se promener localement autour de leur domicile, simulant la vie de quartier.

5. Cycle du soir

Le soir (17h-23h), les agents rentrent progressivement chez eux, avec une légère probabilité de sorties nocturnes pour les actifs, avant que la nuit ne fige la population à domicile.

Cette structure garantit que les mouvements de foule émergent de milliers de décisions individuelles, créant quelque chose de réaliste pour la propagation du virus.

2. La propagation de l'épidémie

Notre simulation implémente une variante du modèle classique SIR, étendue pour inclure les états d'exposition, de décès et de vaccination (**SEIRDV**).

Chaque agent possède un état de santé qui évolue :

S (Susceptible) : L'état par défaut. (**COULEUR BLEU**)

E (Exposed) : L'agent a contracté le virus mais n'est pas encore contagieux (période d'incubation de 3 jours, définie par EPI_INCUBATION). (**COULEUR ROUGE CLAIR**)

I (Infected) : L'agent est contagieux et symptomatique (durée de 10 jours, définie par EPI_DURATION). (**COULEUR ROUGE**)

R (Recovered) : L'agent a survécu et est immunisé à vie. (**COULEUR JAUNE**)

D (Dead) : L'agent décède (probabilité fixe de 3% définie par EPI_MORTALITY, triplée pour les personnes fragiles). (**COULEUR NOIR**)

La propagation du virus est simulée par une boucle d'interactions spatiales exécutée à chaque frame (60 fois par seconde). Dans un premier temps, nous isolons les agents **infectieux (I)** et **susceptibles (S)** qui ne se déplacent pas en véhicule (pour simuler la promiscuité dans les bâtiments ou la marche). Puis pour chaque **paire (I, S)**, nous calculons la distance euclidienne carrée (d^2) pour optimiser les performances en évitant le calcul coûteux de la racine carrée.

Si $d^2 < \text{EPI_RADIUS}^2$ (rayon de 20 pixels), un tirage de Bernoulli est effectué avec une probabilité $p = 0.0025$. Ce taux par frame correspond mathématiquement à une probabilité de contamination $P(t)$ sur t secondes : $P(t) = 1 - (1-p)^{(60t)}$.

Reproduction du Virus (R_0) :

La simulation calcule en temps réel le taux de reproduction effectif (R_t). Chaque agent infecté maintient un compteur `infections_caused`. Le R_t instantané est la moyenne arithmétique de ces compteurs pour tous les agents contagieux actifs, permettant de visualiser dynamiquement l'accélération ou la décélération des vagues épidémiques.

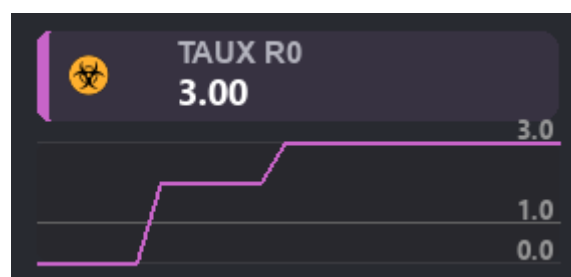


Illustration 3: Le taux R_0 varie en temps réel pendant la simulation

3. L'interface de contrôle – Barre d'action latérale

Cette interface permet d'interagir en temps réel avec les paramètres de la simulation.

Contrôle temporel

Gestion de la vitesse de simulation (x0.5 à x10.0) modifiant le delta temporel appliqué à chaque frame. Cela permet d'accélérer les jours pour observer les tendances de fond ou de ralentir pour analyser les micro-déplacements.

Visualisation des données

Le bouton « Statistiques » ouvre un panneau détaillé affichant les compteurs temps réel pour chaque catégorie (SEIRDV).

Le bouton « Graphe » affiche en surimpression le réseau de nœuds et d'arêtes utilisé par l'algorithme de pathfinding (Visualisation du graphe de navigation).

Vaccination et heatmap

La partie « Vaccination » permet à l'utilisateur de définir un quota journalier de doses (daily_doses). Ces doses sont distribuées aléatoirement chaque matin aux agents éligibles (Sains ou Rétablis), forçant leur transition vers l'état V (Vaccinated) et modifiant ainsi dynamiquement l'immunité.

Le bouton "Heatmap" active une couche de visualisation. À chaque nouvelle infection, les coordonnées de l'événement sont enregistrées. En activant ce mode, la simulation affiche un nuage de points rouges persistants, permettant de cartographier les zones à haut risque (foyers d'infection) et de visualiser l'historique spatial de la propagation.

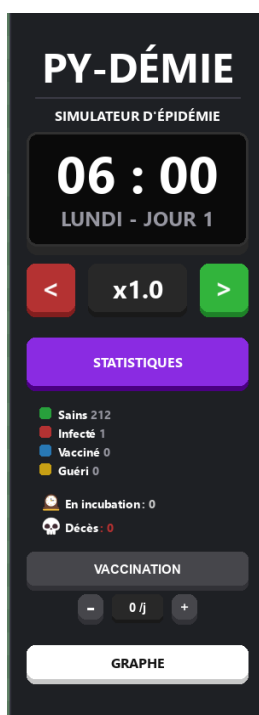


Illustration 5: Barre d'action

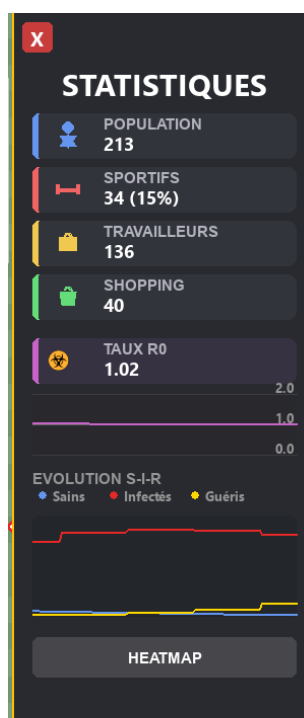


Illustration 4: Panneau stats

Les éléments manquants

Il reste encore plusieurs éléments que nous n'avons pas eu le temps de finaliser. Il était prévu de terminer ces fonctionnalités durant la première semaine de février.

Tout d'abord, il manque un menu permettant de **personnaliser la maladie**. L'idée était de pouvoir modifier différents paramètres, comme le niveau d'agressivité du virus, la vitesse de propagation, ou encore l'apparition de symptômes spécifiques. Cela aurait permis de comparer plusieurs scénarios d'épidémie et d'observer l'impact direct de ces paramètres sur l'évolution de la simulation, tout en rendant le projet plus interactif.

L'idée était vraiment de pouvoir faire intervenir directement le joueur sur la simulation, en lui donnant la possibilité de modifier la maladie au fur et à mesure de la simulation.

Ensuite, nous souhaitions profiter du temps supplémentaire pour **mieux adapter l'interface à toutes les tailles d'écran**. En l'état actuel, selon la résolution utilisée, la barre latérale peut apparaître de manière imparfaite. Une meilleure gestion du redimensionnement aurait amélioré le confort d'utilisation.

Enfin, nous aurions aimé **approfondir l'aspect graphique de la carte**. Notamment en rajoutant du décor supplémentaire. Même si cela reste secondaire par rapport aux aspects mathématiques du projet, une carte plus détaillée aurait rendu le projet encore un peu plus sympa visuellement.